Sorting - Advanced Concepts

Topic: Algorithms

Heapsort

- **Principle:** Heapsort is a comparison-based sorting algorithm that uses a binary heap data structure. It works by first building a max-heap from the input array. Then, it repeatedly extracts the maximum element from the heap (which is always the root) and places it at the end of the array, effectively sorting the array in-place.
- **Stability:** Unstable
- **In-place:** Yes
- **Time Complexity:** O(N log N) in all cases (worst, average, best), making it one of the most reliable efficient sorts.

```
def heapify(arr, n, i):
    largest = i          # Initialize largest as root
    l = 2 * i + 1        # left child
    r = 2 * i + 2        # right child

    # See if left child of root exists and is greater than root
    if l < n and arr[i] < arr[l]:
        largest = l

    # See if right child of root exists and is greater than root
    if r < n and arr[largest] < arr[r]:
        largest = r

    # Change root, if needed
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i] # Swap
        heapify(arr, n, largest) # Heapify the root.

def heap_sort(arr):
    n = len(arr)

    # Build a maxheap.
    # Last parent index is (n//2 - 1)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # One by one extract elements
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i] # Swap root (max) with last
element
        heapify(arr, i, 0) # call heapify on the reduced heap
```

Non-Comparison Sorting Algorithms

1. Counting Sort

- **Principle:** Works by counting the number of occurrences of each distinct element in the input array. It then calculates the position of each element in the output array based on these counts. It is efficient for sorting integers when the range of numbers is not significantly larger than the number of items to be sorted.
- **Stability:** Stable
- **In-place:** No (requires O(Range) auxiliary space)
- **Time Complexity:** O(N + Range), where Range is the difference between max and min element.

```
def counting_sort(arr):
    max_val = max(arr)
    min_val = min(arr)
    range_of_elements = max_val - min_val + 1

    # Create a count array to store count of individual elements
    count = [0] * range_of_elements
    output = [0] * len(arr)

    # Store count of each character
    for i in range(len(arr)):
                    count[arr[i] - min_val] += 1

    # Change count[i] so that count[i] now contains actual
    # position of this character in output array
    for i in range(1, len(count)):
        count[i] += count[i - 1]

    # Build the output array
    for i in range(len(arr) - 1, -1, -1):
        output[count[arr[i] - min_val] - 1] = arr[i]
        count[arr[i] - min_val] -= 1

    # Copy the output array to arr, so that arr now contains sorted
elements
    for i in range(len(arr)):
        arr[i] = output[i]
```

2. Radix Sort

- **Principle:** Sorts elements by processing individual digits. It works by sorting the input array by each digit, from the least significant digit (LSD) to the most significant digit (MSD), using a stable sorting algorithm (like counting sort) for each digit.

- **Stability:** Stable
- **In-place:** No (requires O(N + Base) auxiliary space, where Base is the number of possible digit values)
- **Time Complexity:** O(d * (N + Base)), where d is the number of digits/passes, N is the number of elements, and Base is the radix (e.g., 10 for decimal, 2 for binary).

```
def counting_sort_for_radix(arr, exp):
    n = len(arr)
    output = [0] * n
    count = [0] * 10 # For digits 0-9

    # Store count of occurrences in count[]
    for i in range(n):
        index = (arr[i] // exp) % 10
        count[index] += 1

    # Change count[i] so that count[i] now contains actual
    # position of this digit in output array
    for i in range(1, 10):
        count[i] += count[i - 1]

    # Build the output array
    i = n - 1
    while i >= 0:
        index = (arr[i] // exp) % 10
        output[count[index] - 1] = arr[i]
        count[index] -= 1
        i -= 1

    for i in range(n):
        arr[i] = output[i]

def radix_sort(arr):
    max_val = max(arr)

    # Do counting sort for every digit. exp is 10^i
    exp = 1
    while max_val // exp > 0:
```

```
        counting_sort_for_radix(arr, exp)
        exp *= 10
```

Note: This is a manually-generated study resource for CS Engineering students.# Sorting - Advanced Concepts